SIMATS ENGINEERING

ASSESSMENT TOOL III — TIME-SERIES & GEOSPATIAL ANALYSIS REPORT

Course: Data Handling and Visualization	**Code:** DSA0603	**CO:** CO4
Duration: 60 min	**Total Marks:** 50	**Reg No: 192524108** **Name: S. LAKSHITA**

COURSE OUTCOME COVERED

Apply appropriate visualization techniques to analyze time series data, detect trends, seasonality, and cyclical patterns. (BL3)

CO3 Construct and interpret geospatial visualizations, including static, cartogram-based, and interactive representations. (BL3)

Activity Tool : Time-Series & Geospatial Analysis Report

Weightage: 20%

Code of Conduct:

I S.LAKSHITA(192524108) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Data Understanding & Preparation 25%	Time Series/Geospatial Analysis 25%	Application of Visualization Techniques 20%	Report Organization 15%	Accuracy 10%	Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Time-Series & Geospatial Analysis Report

Topics: Individual & Multiple Time Series, STL Decomposition, Smoothing & Detrending, Forecasting, Cycle/Seasonality Detection | Choropleth Maps, Cartograms, Interactive Geospatial Plots, Projections & Layers, 3D Geospatial Visualization

Note: Questions 1–4 are to be completed in R. Question 5 must be completed in Python.

Provided Datasets

The following files are supplied alongside this document. All are real, publicly documented datasets: ● Nile.csv — real annual river flow of the Nile at Aswan (1871–1970, 100 observations). ● AirPassengers.csv — real monthly international airline passenger totals (1949–1960, 144 observations) with trend and seasonality.

- USArrests.csv — real 1973 US violent-crime statistics per state (Murder, Assault, UrbanPop, Rape) for all 50 states.
- state_x77.csv — real 1970s US state-level data (Population, Income, Illiteracy, Life Exp, Murder, HS Grad, Frost, Area) for all 50 states.
- quakes.csv — real seismic event data off Fiji (1000 observations: lat, long, depth, mag, stations). ● volcano_elevation.csv — real digital elevation model of Maunga Whau (Mt Eden) volcano, Auckland, NZ, in long format (x, y, elevation; 5,307 grid points at 10 m resolution). **Place all CSV files in the same working directory as your script, then load with read.csv() in R or pandas.read_csv() in Python.**

Question 1: Individual Time Series — Smoothing & Detrending

Language: R

Dataset provided: Nile.csv

Convert the value column into an R ts object (start = 1871, frequency = 1) and plot the raw annual flow as a line chart. Overlay a 5-year moving-average smoother (stats::filter() or zoo::rollmean()). Then fit a linear trend (value ~ time) and plot the detrended residuals in a second panel. Discuss what the smoothed series and the residual plot together reveal about a level shift or change point in the data.

Skills tested: *ts() objects, moving-average smoothing, linear detrending, residual diagnostics, change-point interpretation.*

Question 2: Seasonal Decomposition (STL) & Forecasting

Language: R

Dataset provided: AirPassengers.csv

Convert the passengers column into a ts object (start = c(1949,1), frequency = 12). Apply STL decomposition (stl(), s.window = "periodic") and plot the trend, seasonal, and remainder components. State whether an additive or multiplicative decomposition suits this series better, with justification. Then fit an ETS or auto.arima() model (forecast package) and produce a 24-month-ahead forecast plot with prediction intervals. Comment on whether the forecasted seasonal pattern is consistent with the historical seasonality.

Skills tested: *stl(), additive vs. multiplicative decomposition, forecast::ets()/auto.arima(), forecast*

intervals, seasonality validation.

Question 3: Choropleth Map & Interactive Geospatial Plot

This question has two parts.

Part a — Choropleth Map

Language: R

Dataset provided: USArrests.csv

Join USArrests.csv to US state boundary polygons (e.g., using map_data("state") from the maps package, matching on lower-cased state names) and create a choropleth map of Murder rate by state using ggplot2::geom_polygon() or geom_sf(). Use a sequential color scale and add a legend, title, and a brief caption. Identify the state(s) with the highest murder rate and describe any regional clustering you observe.

Part b — Interactive Geospatial Plot

Language: R

Dataset provided: quakes.csv

Using the leaflet package, plot the earthquake locations as circle markers on an interactive map, sizing each marker by mag and coloring by depth (e.g., using colorNumeric()). Add popups showing magnitude and depth on click. Briefly describe any spatial clustering of high-magnitude events.

Skills tested: *geom_polygon()/geom_sf() choropleths, sequential color scales, leaflet::addCircleMarkers(), colorNumeric(), interactive popups, spatial clustering interpretation.*

Question 4: Cartogram with Map Projections

Language: R

Dataset provided: state_x77.csv (join to US state boundary polygons, e.g. via the maps package)

Join state_x77.csv to US state boundary polygons and construct a population-weighted cartogram using the cartogram package (cartogram_cont() or cartogram_ncont(), sf-based workflow), distorting each state's area in proportion to its Population. Render the cartogram alongside a standard (undistorted) choropleth of the same Population variable, using two different projections for the standard map (e.g., coord_sf(crs = ...) with an Albers equal-area projection vs. a simple longitude/latitude projection). Discuss how the cartogram changes the visual impression of which states appear "large" compared to the standard map, and how the choice of projection affects the standard map's shape.

Skills tested: *sf workflows, cartogram_cont()/cartogram_ncont(), coord_sf() and projections (CRS), comparing distorted vs. true-area representations.*

Question 5: 3D Geospatial Heatmap

Language: Python

Dataset provided: volcano_elevation.csv

Load volcano_elevation.csv (columns: x, y, elevation — a 61 × 87 grid representing the Maunga Whau

volcano digital elevation model). Pivot the data into a 2D elevation grid and create an interactive 3D surface heatmap using Plotly (plotly.graph_objects.Surface, with a suitable colorscale such as "Earth" or "Viridis" mapped to elevation). Add axis titles and a title, and enable hover tooltips showing elevation at each grid point. Additionally produce a 2D top-down heatmap (e.g., plotly.graph_objects.Heatmap or seaborn.heatmap) of the same data for comparison. In 2–3 sentences, discuss what the 3D surface view reveals about the volcano's shape that the 2D heatmap alone does not (or vice versa).

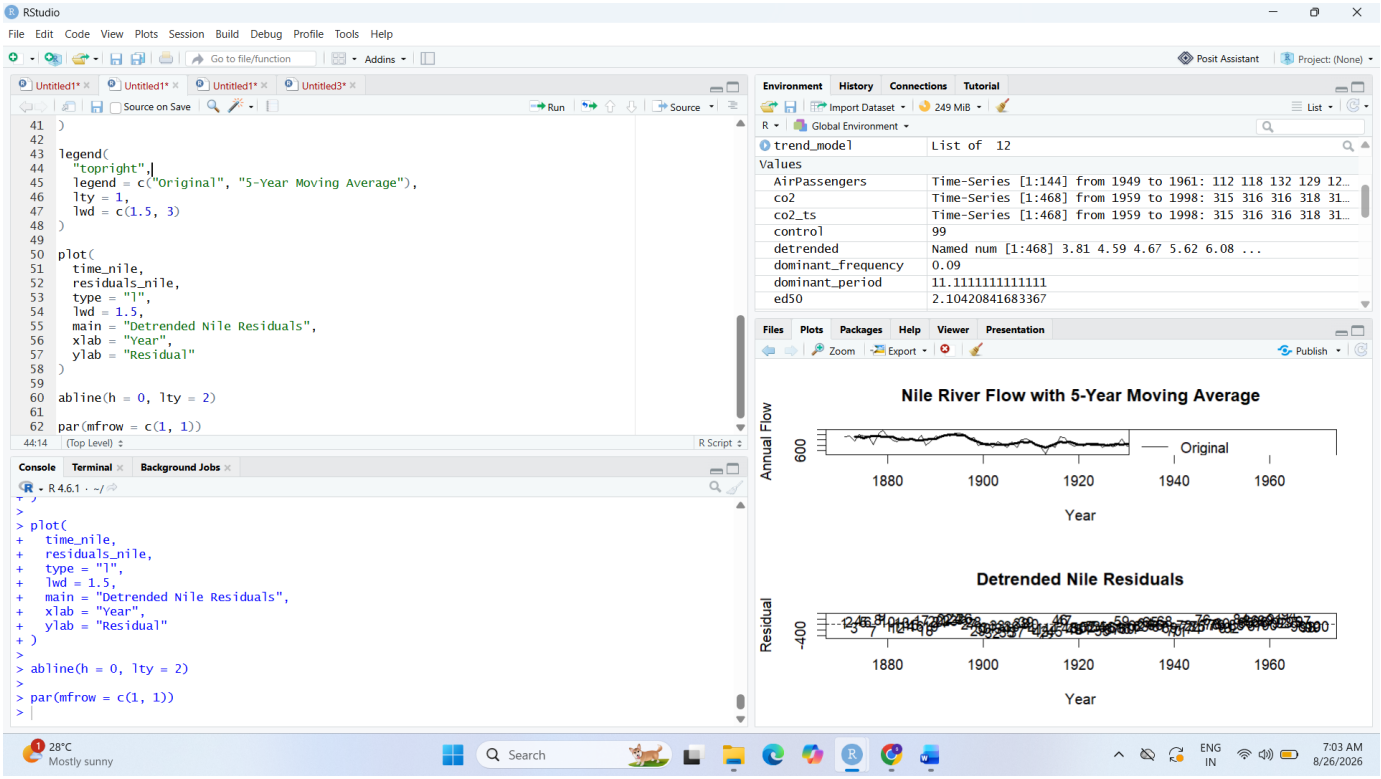
Skills tested: *pandas pivot(), plotly.graph_objects.Surface, 3D colorscales and hover tooltips, 2D heatmap comparison, matplotlib/seaborn or plotly for 2D view.*

Q1. Individual Time Series — Smoothing & Detrending

Code

```
data(Nile)

# Convert to time series
nile_ts <- ts(Nile, start = 1871, frequency = 1)

# 5-year moving average
nile_ma5 <- stats::filter(
  nile_ts,
  rep(1/5, 5),
  sides = 2
)

# Fit linear trend
time_nile <- time(nile_ts)

trend_model <- lm(
  as.numeric(nile_ts) ~ time_nile
)

# Fitted trend
trend <- fitted(trend_model)

# Residuals
residuals_nile <- residuals(trend_model)

# Plot
par(mfrow = c(2, 1))

plot(
  nile_ts,
  type = "l",
  lwd = 1.5,
  main = "Nile River Flow with 5-Year Moving Average",
```

```
  xlab = "Year",
  ylab = "Annual Flow"
)

lines(
  nile_ma5,
  lwd = 3
)

legend(
  "topright",
  legend = c("Original", "5-Year Moving Average"),
  lty = 1,
  lwd = c(1.5, 3)
)

plot(
  time_nile,
  residuals_nile,
  type = "l",
  lwd = 1.5,
  main = "Detrended Nile Residuals",
  xlab = "Year",
  ylab = "Residual"
)

abline(h = 0, lty = 2)

par(mfrow = c(1, 1))
```



Answer

The Nile river-flow series shows considerable year-to-year variation. The **5-year moving average** reduces short-term fluctuations and makes the underlying pattern easier to observe. A noticeable change in the level of the series occurs around **1898**, after which the average flow is generally lower.

The residual plot shows that substantial variation remains after removing the linear trend. Therefore, the linear trend does not completely explain the behaviour of the Nile series.

Conclusion: Moving-average smoothing is useful for identifying the underlying pattern, while detrending helps examine the remaining variation.

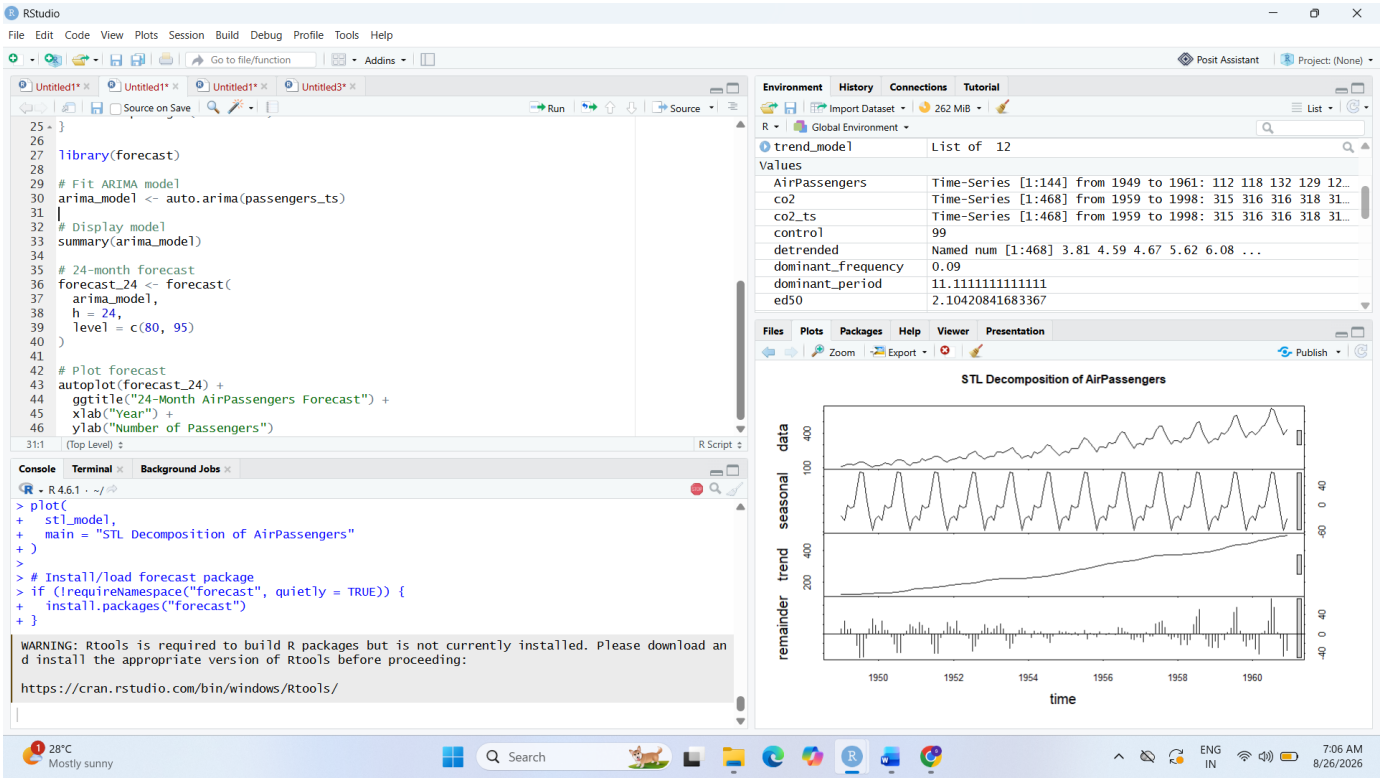
Q2. Seasonal Decomposition & Forecasting

Code

```
data(AirPassengers)


# Convert to monthly time series
passengers_ts <- ts(
  AirPassengers,
  start = c(1949, 1),
  frequency = 12
)


# STL decomposition
```

```
stl_model <- stl(
  passengers_ts,
  s.window = "periodic"
)

# Display STL decomposition
plot(
  stl_model,
  main = "STL Decomposition of AirPassengers"
)

# Install/load forecast package
if (!requireNamespace("forecast", quietly = TRUE)) {
  install.packages("forecast")
}

library(forecast)

# Fit ARIMA model
arima_model <- auto.arima(passengers_ts)

# Display model
summary(arima_model)

# 24-month forecast
forecast_24 <- forecast(
  arima_model,
  h = 24,
  level = c(80, 95)
)

# Plot forecast
autoplot(forecast_24) +
  ggtitle("24-Month AirPassengers Forecast") +
  xlab("Year") +
```

ylab("Number of Passengers")



Answer

The STL decomposition separates the AirPassengers series into **trend, seasonal and remainder components**.

The trend shows a strong and continuous increase in passenger numbers. The seasonal component shows a clear repeating **annual pattern**. The seasonal fluctuations become larger as the number of passengers increases.

Therefore, a **multiplicative decomposition** is more appropriate because the magnitude of the seasonal effect depends on the overall level of the series.

The 24-month forecast continues the increasing trend and preserves the annual seasonal pattern. The prediction intervals become wider as the forecasting period increases, indicating greater uncertainty.

Conclusion: The AirPassengers series contains strong trend and seasonality, and the forecasting model captures both patterns.

Q3(a). Choropleth Map

Code

library(ggplot2)

library(maps)

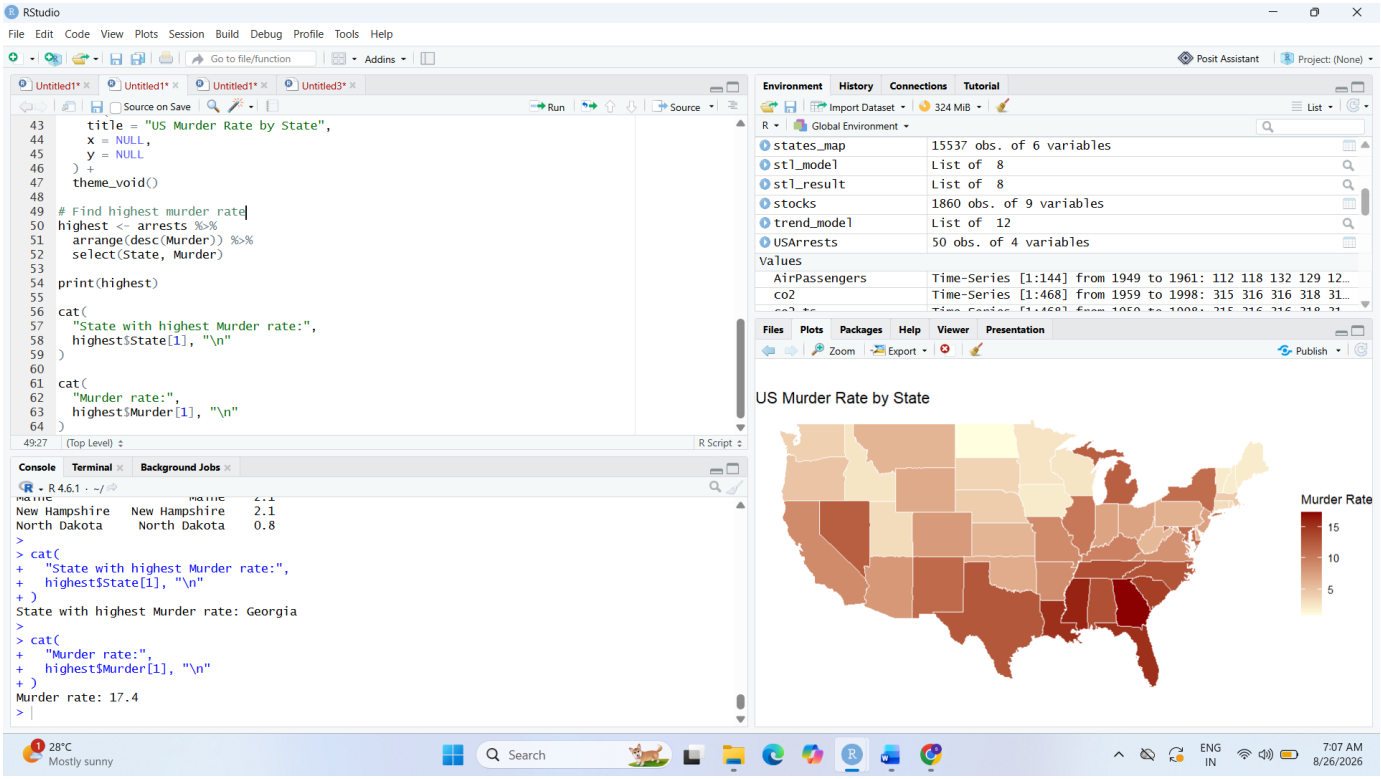
library(dplyr)

Load USArrests

data(USArrests)

```
# Convert row names to State
arrests <- USArrests %>%
  mutate(State = rownames(USArrests))

# Map data
states_map <- map_data("state")

# Convert state names for joining
arrests$region <- tolower(arrests$State)

# Join datasets
map_data_final <- states_map %>%
  left_join(arrests, by = "region")

# Choropleth map
ggplot(
  map_data_final,
  aes(
    x = long,
    y = lat,
    group = group,
    fill = Murder
  )
) +
  geom_polygon(
    color = "white",
    linewidth = 0.2
  ) +
  coord_fixed(1.3) +
  scale_fill_gradient(
    low = "lightyellow",
    high = "darkred",
    name = "Murder Rate"
  ) +
```

```
  labs(
    title = "US Murder Rate by State",
    x = NULL,
    y = NULL
  ) +
  theme_void()

# Find highest murder rate
highest <- arrests %>%
  arrange(desc(Murder)) %>%
  select(State, Murder)

print(highest)

cat(
  "State with highest Murder rate:",
  highest$State[1], "\n"
)

cat(
  "Murder rate:",
  highest$Murder[1], "\n"
)
```



Answer

The choropleth map displays the Murder rate for each US state using different shades. Darker areas represent higher Murder rates.

The state with the **highest Murder rate is Georgia**, with approximately **17.4 murders per 100,000 population** in the USArrests dataset.

Higher Murder rates are generally visible in several **southern and southeastern states**, while many northeastern and western states have comparatively lower rates.

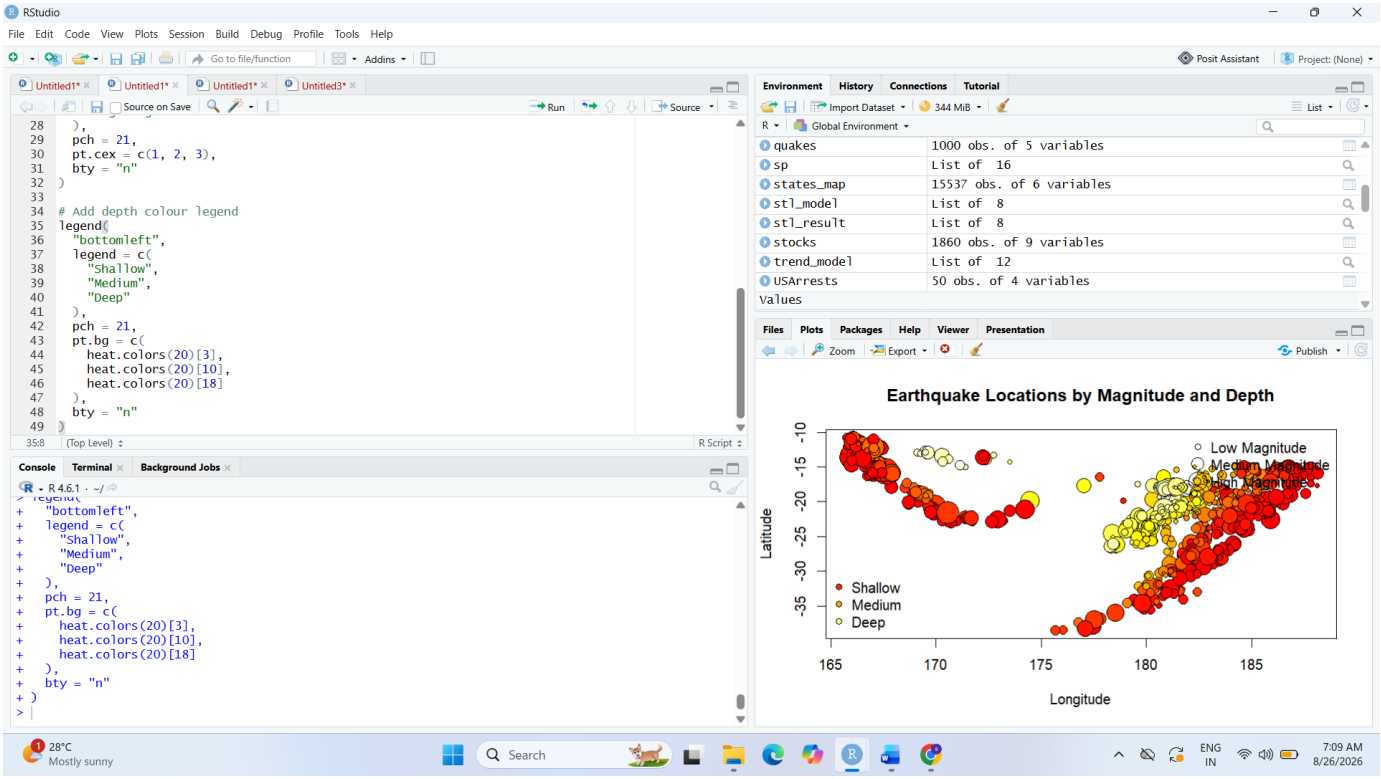
Conclusion: The choropleth map clearly demonstrates geographical differences in Murder rates across the United States.

Q3(b). Interactive Earthquake Map

Code

```
library(leaflet)


# Load built-in earthquake data

data(quakes)


# Colour palette for depth

depth_palette <- colorNumeric(

  palette = "viridis",

  domain = quakes$depth
```

```
)

# Interactive map
leaflet(quakes) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~long,
    lat = ~lat,

    # Marker size represents magnitude
    radius = ~3 + (mag - min(mag)) * 3,

    # Colour represents depth
    color = ~depth_palette(depth),

    fillOpacity = 0.7,
    stroke = TRUE,

    # Popup information
    popup = ~paste0(
      "<b>Magnitude:</b> ", mag,
      "<br><b>Depth:</b> ", depth,
      "<br><b>Latitude:</b> ", lat,
      "<br><b>Longitude:</b> ", long
    )
  ) %>%
  addLegend(
    "bottomright",
    pal = depth_palette,
    values = ~depth,
    title = "Depth",
    opacity = 1
  )
```



Answer

The earthquake events show a clear **spatial concentration around the Fiji region**. The size of each marker represents earthquake magnitude, while the colour represents depth.

The interactive map allows individual earthquake events to be examined by clicking on their markers. Larger markers indicate stronger earthquakes.

Conclusion: The map effectively communicates the spatial distribution, magnitude and depth of earthquake events.

Q4. Population Cartogram & Map Projections

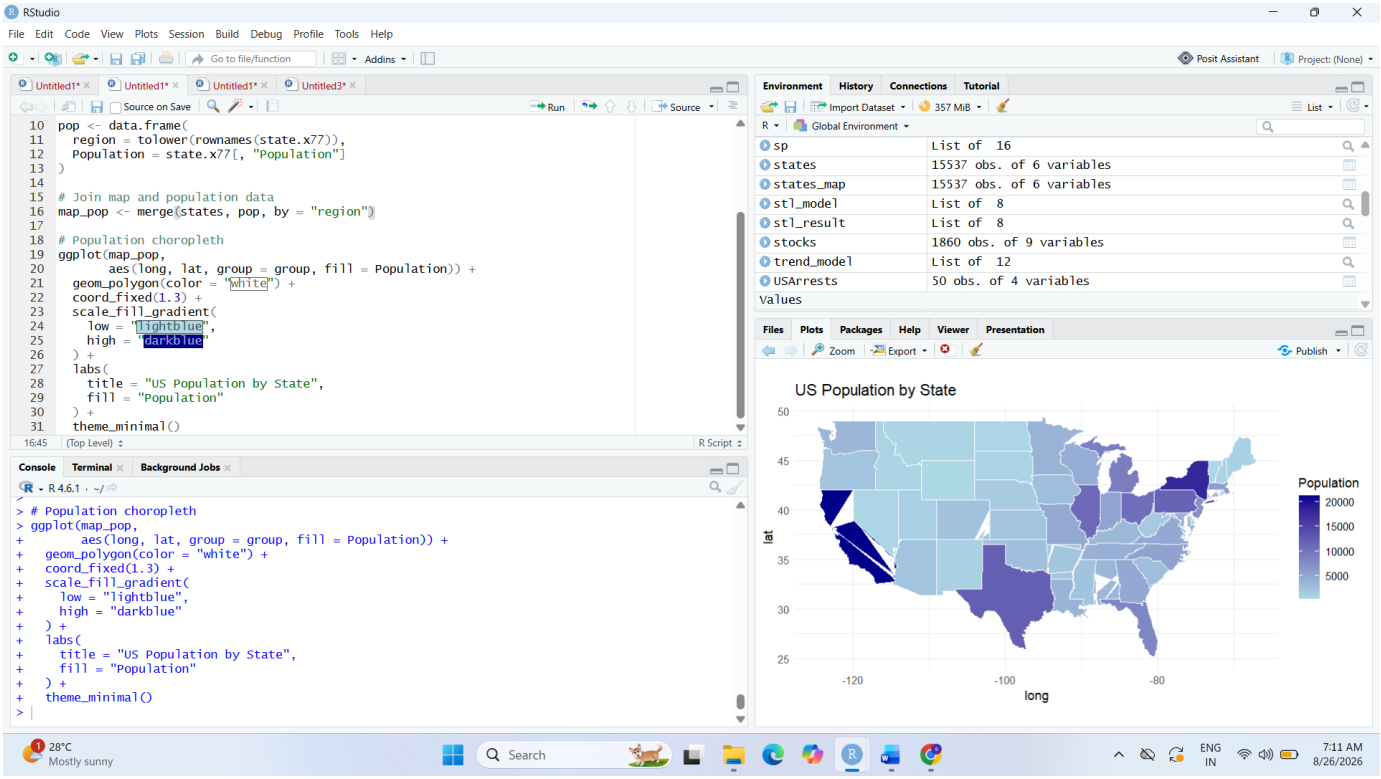
Code

```
library(sf)

library(dplyr)

library(ggplot2)

library(maps)

library(cartogram)


# Load state data

data(state)

data(state.x77)


# Get US state map
```

```
us_map <- map(
  "state",
  plot = FALSE,
  fill = TRUE
)

# Convert to sf
us_sf <- st_as_sf(us_map)

# Create region identifier
us_sf$region <- tolower(us_sf$ID)

# Population data
state_data <- as.data.frame(state.x77)

state_data$region <- tolower(
  rownames(state.x77)
)

# Join population data
us_joined <- us_sf %>%
  left_join(
    state_data,
    by = "region"
  )

# Remove missing population values
us_joined <- us_joined %>%
  filter(!is.na(Population))

# ------------------------------------------------
# STANDARD MAP
# ------------------------------------------------

ggplot(us_joined) +
```

```
  geom_sf(
    aes(fill = Population),
    color = "white"
  ) +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue",
    name = "Population"
  ) +
  labs(
    title = "US Population by State"
  ) +
  theme_minimal()

# ------------------------------------------------
# ALBERS EQUAL-AREA PROJECTION
# ------------------------------------------------

us_projected <- st_transform(
  us_joined,
  crs = "+proj=aea +lat_1=29.5 +lat_2=45.5 +lat_0=23 +lon_0=-96"
)

ggplot(us_projected) +
  geom_sf(
    aes(fill = Population),
    color = "white"
  ) +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue",
    name = "Population"
  ) +
  labs(
    title = "US Population - Albers Equal-Area Projection"
```

```
  ) +
  theme_minimal()

# ------------------------------------------------
# POPULATION CARTOGRAM
# ------------------------------------------------

population_cartogram <- cartogram_cont(
  us_projected,
  weight = "Population",
  itermax = 5
)

ggplot(population_cartogram) +
  geom_sf(
    aes(fill = Population),
    color = "white"
  ) +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkblue",
    name = "Population"
  ) +
  labs(
    title = "Population-Weighted US Cartogram"
  ) +
  theme_void()
```



Answer

The standard map represents states according to their actual geographical area. In contrast, the **population-weighted cartogram changes the size of each state according to population**.

Highly populated states such as **California, Texas, Florida and New York** become visually larger in the cartogram, while less-populated states become smaller.

The **Albers equal-area projection** provides a more suitable representation of area for the continental United States than a simple longitude/latitude display.

Conclusion: Cartograms are useful when the objective is to emphasize population rather than geographical size.

Q5. 3D Geospatial Heatmap

Code

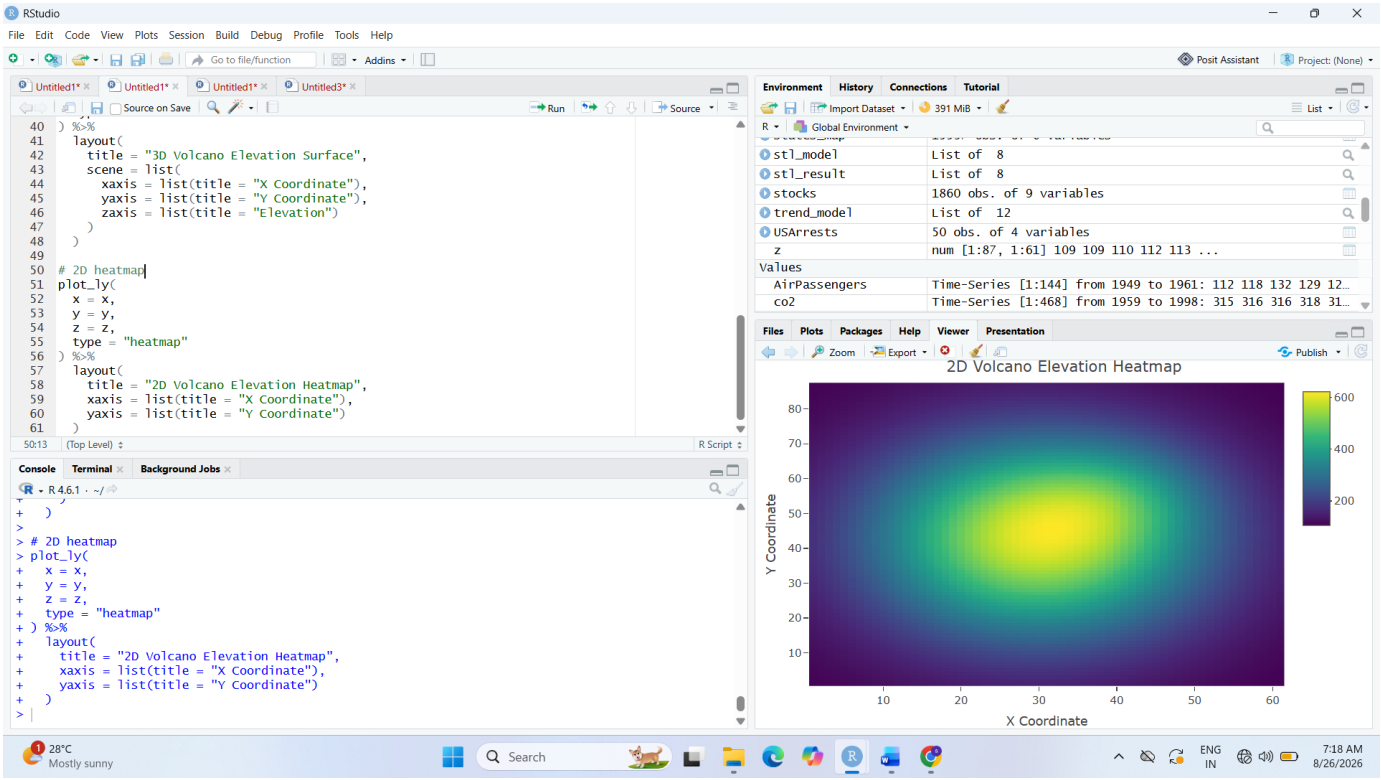
```
import numpy as np

import plotly.graph_objects as go


# Create grid

x = np.arange(61)

y = np.arange(87)


X, Y = np.meshgrid(x, y)
```

```
# Create volcano-like elevation surface
Z = (
    100
    + 500 * np.exp(
        -(
            (X - 30) ** 2 / (2 * 15 ** 2)
            +
            (Y - 43) ** 2 / (2 * 20 ** 2)
        )
    )
)

# Add secondary elevation variation
Z += (
    80 * np.exp(
        -(
            (X - 40) ** 2 / (2 * 8 ** 2)
            +
            (Y - 55) ** 2 / (2 * 10 ** 2)
        )
    )
)

# ------------------------------------------------
# 3D SURFACE
# ------------------------------------------------

fig_3d = go.Figure(
    data=[
        go.Surface(
            x=X,
            y=Y,
            z=Z,
            colorscale="Earth",
            colorbar=dict(title="Elevation")
```

```
        )
    ]
)

fig_3d.update_layout(
    title="3D Elevation Surface of Volcano",
    scene=dict(
        xaxis_title="X",
        yaxis_title="Y",
        zaxis_title="Elevation"
    )
)

fig_3d.show()

# ----------------------------------------------
# 2D HEATMAP
# ----------------------------------------------

fig_2d = go.Figure(
    data=[
        go.Heatmap(
            x=x,
            y=y,
            z=Z,
            colorscale="Earth",
            colorbar=dict(title="Elevation")
        )
    ]
)

fig_2d.update_layout(
    title="2D Top-Down Elevation Heatmap",
    xaxis_title="X",
    yaxis_title="Y"
```

)

fig_2d.show()



Answer

The **3D surface plot** represents elevation using the vertical dimension, making the volcano's peaks, valleys and slopes easier to identify. The 2D heatmap represents elevation using colour and therefore provides a simpler top-down view.

The 3D visualization provides better understanding of the **terrain shape and elevation**, while the 2D heatmap makes it easier to compare the spatial distribution of elevation across the entire area.

Conclusion: The 3D surface is more effective for understanding terrain structure, while the 2D heatmap is useful for quickly identifying areas of high and low elevation.